

Course Name	Big Data Systems Design
Course Code	MBAI 5110 G
Term & Year	Winter 2026
Assignment Number	2
Submission Due Date	2026/02/27
Submitted Date	2026/02/26
Student Name	Afeena Gafoor
Student ID	101018679

Table of Contents

Introduction 3

Problem Statement 4

Proposal 5

E-Commerce Platform Architecture 6

E-Commerce Big Data Architecture Explanation 7

Big Data Storage Design 8

 Identify the data types of the e-commerce platform 8

 Design a storage architecture using Hadoop Distributed File System (HDFS) and Hive for structured querying 8

 Explain data partitioning, replication, and fault-tolerance in HDFS..... 8

 Discuss when to use HBase for real-time read/write access and Hive/Parquet for analytics. 10

 Include a diagram showing data storage layers and interaction between HDFS, Hive, and HBase..... 11

Big Data Processing Framework..... 12

Tools & Ecosystem..... 14

Conclusion 15

Appendix A..... 16

Citations..... 17

Introduction

Amazon Inc. is a premier global retail conglomerate generating an annual net profit of **\$100 million**. As the company expands into international markets, its data footprint has reached a critical mass, characterized by high volume, velocity, and variety. Currently, Amazon Inc. relies on a legacy architecture comprised of Relational Database Management Systems (RDBMS) to manage core business entities, including global inventory, multi-currency payment transactions, and complex product catalogs. While this structured approach served initial growth, the infrastructure is now a bottleneck, hindering the company's ability to transition into a truly data-driven enterprise.

Problem Statement

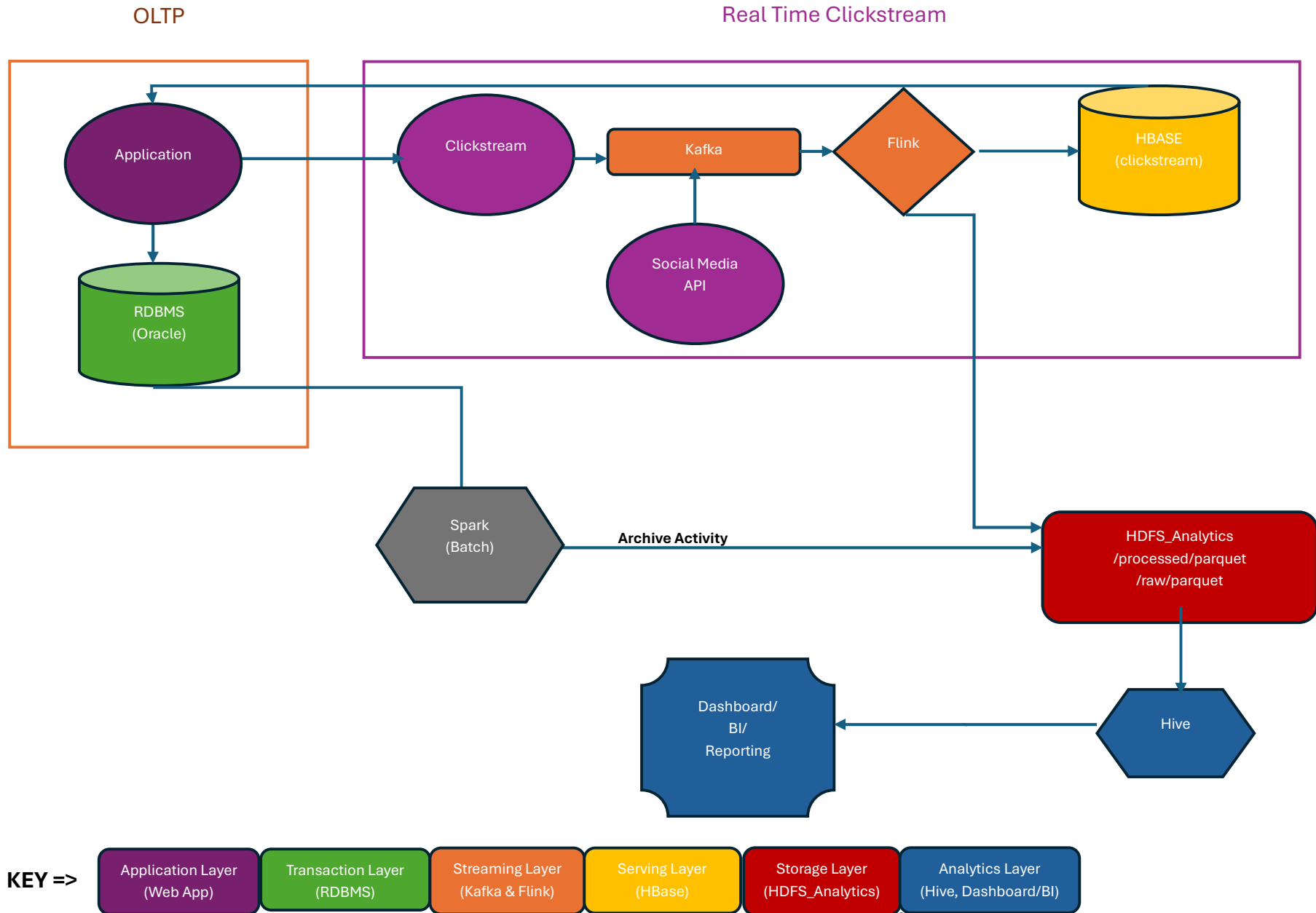
Despite its market success, **Amazoon** Inc.'s technical infrastructure is failing to keep pace with global demand, primarily due to the limitations of traditional relational scaling. The organization faces three critical challenges:

- **Scaling Costs:** The current RDBMS architecture requires **vertical scaling** (expensive hardware upgrades) to manage increasing loads, resulting in a **10% year-over-year increase** in infrastructure expenditure. This model is financially unsustainable for a global conglomerate.
- **Query Latency:** With datasets exceeding **one trillion records**, the lack of logical partitioning and indexing in the legacy RDBMS has caused query performance to degrade significantly. Analytical reports that once took minutes now take hours.
- **Inability to Process Non-Linear Data:** Amazoon Inc. is currently “data-blind” to customer sentiment and real-time behavior. The existing system cannot ingest or analyze unstructured and semi-structured data- such as high-frequency web logs, clickstreams, and social media interactions- preventing the business from delivering a personalized, proactive customer experience.

Proposal

Amazon Inc. is looking for an inexpensive and scalable solution to current issues. We at Afeena Inc. will be provide a design architecture using the Hadoop framework to resolve the issues at a more affordable and sustainable price. This implementation utilises the use of Hadoop to respond to their growth.

E-Commerce Platform Architecture



E-Commerce Big Data Architecture Explanation

The proposed architecture separates transactional processing, real-time analytics, and data warehousing into distinct layers.

The web application interacts directly with an RDBMS, which functions as the OLTP system. This database manages transactional operations such as orders, inventory updates, and product information. The RDBMS ensures ACID compliance and supports day-to-day operational workflows.

Simultaneously, user interactions generate clickstream data, which is captured and published to Apache Kafka. Kafka acts as the streaming ingestion layer, enabling fault-tolerant event collection. We did not choose flume in the event Flink stops working flume does not have the capacity to hold the data. Social media feedback is also ingested as an additional event stream through Kafka and processed in real time using Flink.

For real-time analytics, clickstream events are processed using Apache Flink. Flink performs event-time processing and computes behavioral insights such as related product recommendations. The computed recommendations are written to a serving database- Apache HBase, which provides low-latency key-based lookups. The web application queries this serving layer to instantly return similar items (e.g., other red shoes) when a user performs a search. This design avoids sending recommendation results back to the RDBMS, thereby preventing transactional bottlenecks.

For historical analytics and reporting, both transactional data from the RDBMS and processed clickstream data are stored in Apache Hadoop HDFS in Parquet format. Batch ETL from the RDBMS is performed using Apache Spark, ensuring efficient large-scale transformations.

The analytical layer is implemented using Apache Hive, which provides SQL-based querying over the Parquet datasets stored in HDFS. Business intelligence tools connect to Hive to generate dashboards, reports, and long-term trend analysis.

By separating the architecture into:

- OLTP (RDBMS),
- Real-time streaming (Kafka + Flink),
- Serving layer (HBase),
- Data warehouse storage (HDFS),
- Analytics layer (Hive),

the system ensures low-latency recommendations for users while also supporting scalable historical analytics and reporting.

Big Data Storage Design

Identify the data types of the e-commerce platform

There are two types of data types in this scenario: clickstream and OLTP. Below of a breakdown of how it will be used

- **Application server logs (clickstream data):** Answers the question, “What did the human do”. Logs clickstream behaviour from Amazon Inc. For example, button clicks, hover time, scroll depth, items added to a cart (even if not purchased), and the specific sequence of navigation (e.g., *Home > Search “Shoes”>Click Product X*).
- **Social Media Mentions (clickstream data):** Each social media platform provides endpoints that broadcast real- time streams of public mentions or hashtags related to “Amazon Inc.”.
- **Transactional ERP (OLTP):** Primary source for order details and payment transactions
- **Inventory System (OLTP):** Tracks stock level

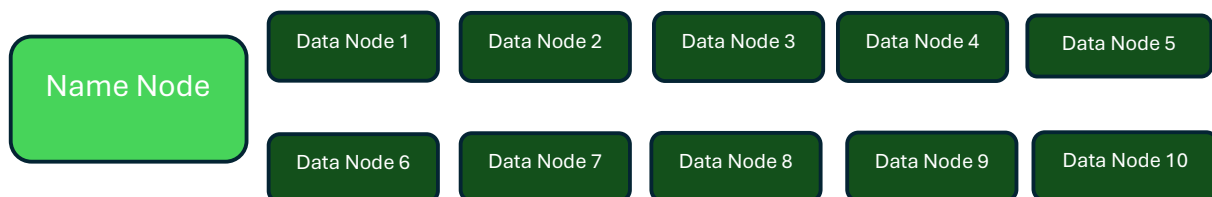
Design a storage architecture using Hadoop Distributed File System (HDFS) and Hive for structured querying

For full architecture see “E-commerce Platform Architecture” diagram on page 6

Explain data partitioning, replication, and fault-tolerance in HDFS.

1. Data partitioning on HDFS:

CLUSTER:



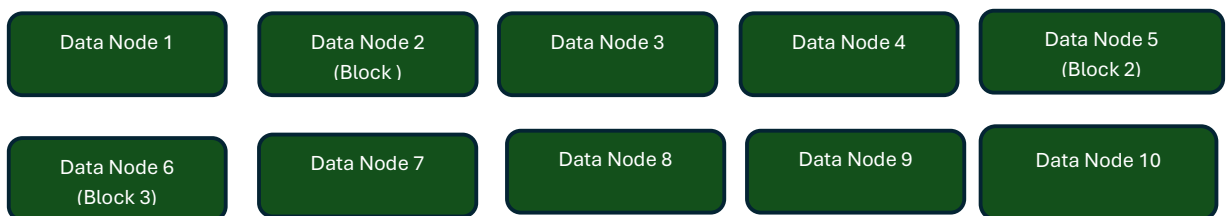
The cluster is viewed through a gateway:



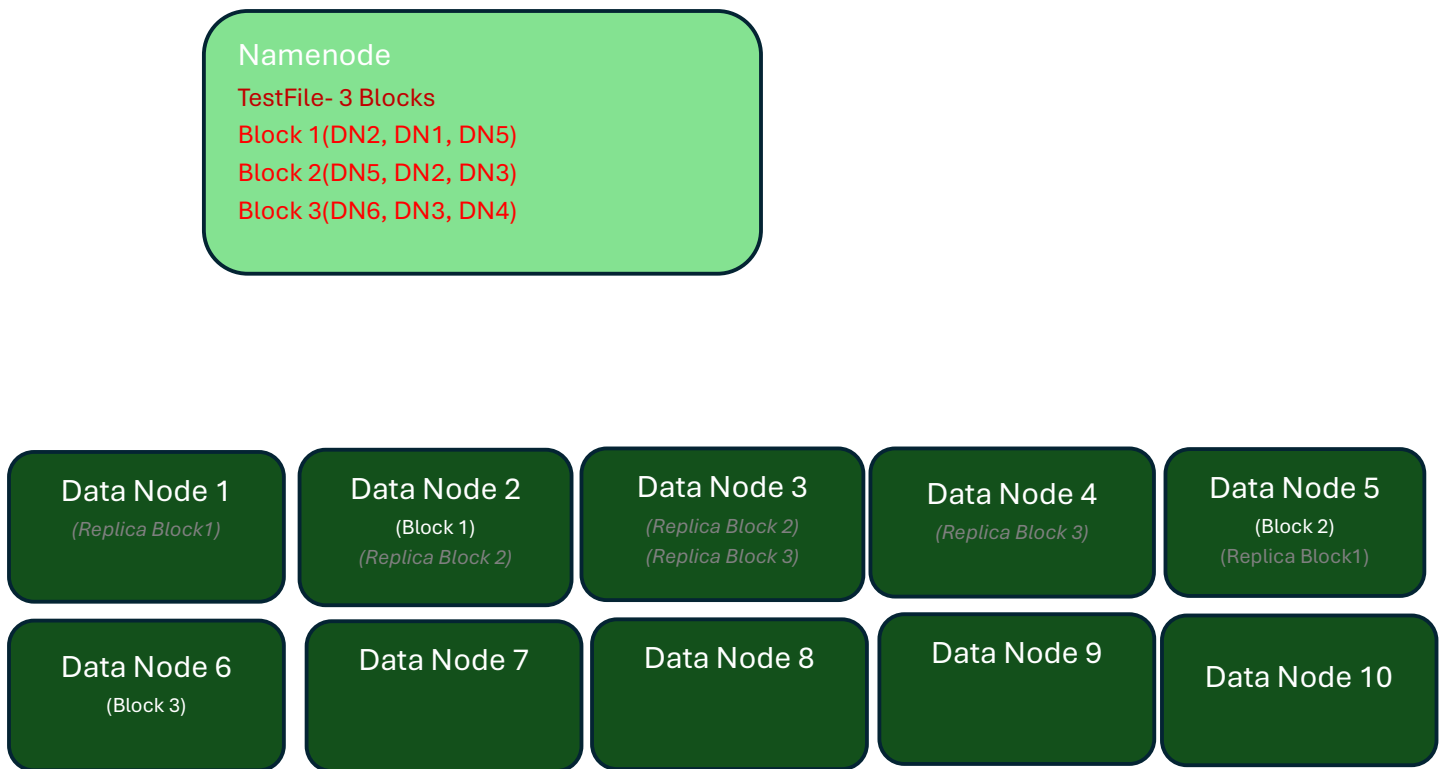


In this scenario a file name “testFile” is 384 MB file is being uploaded.

- **STEP 1 (Determine Block Size):** Read Name node and determine block size. E.g. 128 MB. This is the mx size of the data you can store
- **STEP 2 (Divide File):** The name node tells the package to split the 384MB in three 128MB files. This process is automatic.
- **STEP 3:** After the division is done, the data is then distributed across the data nodes. The Name Node determines where to store the data and records that information. The name node communicates with all 6 of the data nodes and it knows how much space is available. It then determines where to store the blocks. Occasionally the blocks can end up in one data node, but ideally it is stored in different data nodes.



2. **Replication on HDFS:** By default, Hadoop replicates the node on two additional data nodes. The



3. **Fault Tolerance on HDFS:** The name node knows that one of the data nodes is down. If Data Node 2 goes down and the system is look for block 1 data, the name node knows block 1 data can be recovered from Data Node 5 or Data Node 1.

Discuss when to use HBase for real-time read/write access and Hive/Parquet for analytics.

a. HBase:

- i. **Use Case 1:** Personalization of user's browsing history and instantly giving recommendation while they are still on the page. For example, if the user searches for *Red shoes* HBase would provide other red shoe options. HBase provides random, real-time read/write access that HDFS alone cannot handle.
- ii. **Use Case 2:** Updates from multiple vendors can occur frequently, HBase handles frequent row level updates. Updating a single product price in a Parquet file requires rewriting the whole file

b. Parquet:

- i. **Use Case 1:** Generate weekly, monthly sales, reports to users. Parquet's column format allows Hive to skip irrelevant columns (like customer phone number) and

read only “purchase price” and “date”

- ii. **Use Case 2:** Analyzing storing historical sentiment data in Parquet, queries can be run to find correlations identify resonating “Negative Sentiment”

Include a diagram showing data storage layers and interaction between HDFS, Hive, and HBase.

For full architecture see “E-commerce Platform Architecture” diagram on page 6

Big Data Processing Framework

1. **Identify the processing need:** The proposed **Amazon Inc.** architecture will be using both batch and real time streaming to address the different processing needs.
2. **Processing Framework:**
 - a. **Spark Structured Streaming or Apache Flink:** For real time processing Flink was chosen. Spark Structured Streaming uses micro batch processing, while Flink uses true stream processing. Flink generally offers lower latency, while Spark provides batch and streaming support.
 - b. **Apache Spark for Batch and Streaming Analytics:** Apache Spark (for batch) is used to fetch the data from the RDMS into HDFS_Analytics
 - c. **Use MapReduce for legacy batch jobs where appropriate:** Hive will use map reduce on HDFS_Analytics and the results will viewed through the Dashboard/BI/Reporting tool.
 - d. **Explain how fault tolerance is achieved with Spark and HDFS integration:** In the system design, Spark and HDFS achieve fault tolerance through a combination of re-computation (Spark) and redundancy (HDFS). While HDFS focuses on keeping the data safe on disk, Spark focuses on making sure the processing can resume if a node crashes.
 - i. **(Processing) How HDFS handles fault tolerance:** Provides the reliable foundation for your data
 1. *Data Replication:* HDFS splits blocks and stores three copies of each across nodes
 2. *Automatic Recovery:* if a data node fails, the name node detects it and immediately triggers the replication of the blocks into healthy nodes to maintain replication.
 - ii. **(Storage) How Spark handles fault tolerance:** Spark does not replicate data in memory it uses blueprints to rebuild
 1. *Lineage:* The sequence of transformation used to create it from original HDFS source
 2. *Lazy Re-computation:* If an executor fails and a piece of data is lost, Spark refers to this lineage and recomputes only the missing partition by re-running those specific steps.
 3. *Task Retries:* If a task fails on one node, the Spark Driver automatically reschedules it on another healthy node

iii. **Spark Checkpointing to HDFS:** *Spark uses HDFS as a safety net to truncate lineage chain*

1. If your processing chain is very long (e.g., hundreds of transformations), recomputing from the very beginning after a failure is too slow.
2. Reliable Checkpointing: Spark can “checkpoint” intermediate data by saving the current state of an RDD or DataFrame directly to HDFS.
3. Recovery: Once data is saved to HDFS, Spark “forgets” the previous lineage. If a failure occurs later, it simply reads the checkpointed data from HDFS and continues, rather than starting over

e. **Include a data flow/block diagram showing ingestion → storage → processing → analytics pipeline**

For full architecture see “E-commerce Platform Architecture” diagram on page 6

Tools & Ecosystem

The architecture is divided into three main parts:

- 1. Transaction Processing
- 2. Real time Processing
- 3. Data Warehouse Analytics

TRANSACTION LAYER: The web application interacts directly with the RDMS. This database stores products, manages inventory and records orders. This is the OLTP and no streaming is involved here.

REAL TIME BEHAVIOUR PROCESSING. This can be split into the following

- 1. **Event Generation:** The user searches for “red shirt” a click stream event is generated. Let’s say he adds it to the cart, another click stream event occurs.
- 2. **Event Ingestion:** Each event is sent to Apache Kafka. Kafka buffers the event and ensures fault tolerance
- 3. **Stream Processing:** Apache Kafka feeds into Apache Flink. Flink processes the events in real time, performs any filtering or aggregation, computes recommendations (for other red shirts) and can detect trends.
- 4. **Dual Output:** This design used a Fan-Out Pattern (Apache Software Foundation 2025). Flink writes to two destinations
 - a. **Apache HBase (Serving Layer):** Stores processed recommendations and provides quick lookup. The Web App queries HBase directly to return recommendations instantly.
 - b. **HDFS (Analytics Storage):** HDFS stores raw and processed events and log term historical data as parquet files.

DATA WAREHOUSE & ANALYTICS

Data is stored in HDFS and queried using Apache Hive. Hive provides an SQL interface, creates logical tables over parquet and supports aggregation and reporting. The BI tools connect to Hive to generate dashboards.

Table 1- Tools Used by Layer

Layer	Tool	Role
Application	Web App	User interaction
OLTP	RDBMS	Transactions
Streaming Ingestion	Kafka	Event buffering
Stream Processing	Flink	Real-time computation
Serving	HBase	Fast recommendation lookup
Storage	HDFS	Historical storage
Warehouse	Hive	SQL analytics
Visualization	BI Tool	Reporting

Conclusion

This architecture separates transactional processing, real-time streaming, and data warehousing to ensure scalability and performance. The web application relies on the RDBMS for operational transactions, while clickstream events are ingested through Apache Kafka and processed in real time using Apache Flink. Real-time recommendations are stored in Apache HBase for low-latency access, while historical data is persisted in Apache Hadoop HDFS and analyzed using Apache Hive for reporting and business intelligence.

By separating streaming, serving, and analytics layers, the system supports both instant user recommendations and scalable long-term data analysis.

Appendix A

Feature	Spark Structured Streaming	Flink	Spark Batch
Processing Model	Micro batch	True Streaming	Batch processing
Best for	Near real-time analytics + ETL	Real-time event processing, event-time windows	ETL, data warehouse loading, heavy joins
Data Type	Continuous streaming	Continuous event stream	Historical dataset
Latency	Low	Very low	High(minutes/hours)
Batch + Streaming Unified	Yes (unified Spark engine)	Primarily streaming (batch supported but not primary strength)	Batch only

Data Type	Processing Engine	Query Layer
Click stream (real time)	Spark structured streaming	Hive
Click stream (real time)	Flink	Hive
OLTP (batch ETL)	Spark Batch	Hive

Feature	Sqoop (Pull) No support from June 2001	Flume (Push)
Primary use case	Nightly bulk imports from production SQL databases	Continuous aggregation of log files from web servers.
Data structure	Highly structured (Relational data).	Unstructured or semi-structured
Time	Batch based	Real-time
Key Strength	Efficient parallel data transfer	High-volume, distributed, and fault-tolerant
Storage target	HDFS, Hive, or HBase.	HDFS, HBase, or Solr.

Citations

- Apache Software Foundation. (2025, August 20). *HDFS router-based federation* (Version 3.4.2) [Computer software manual]. <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs-rbf/HDFSRouterFederation.html>
- Apache Software Foundation. (2025). *Apache Kafka* (Version 3.9.0) [Computer software]. kafka.apache.org.